

Điểm nổi bật

EMESOFT Agent Suite dùng chung một identity provider và một kho cấu hình: **EmeHub** giữ user, credential và project; **Q-Agent** chạy pipeline QA. Agent khác — D-Agent cho dev, B-Agent cho BA — plug vào qua cùng contract mà không phải sửa hub (\$7).
Mỗi khẳng định dưới đây đều dẫn tới file hoặc ADR trong repo.

1. Claude credential: hai lớp, một switch

Vấn đề: một team dùng chung một Claude account thì ai cũng phải xếp hàng sau rate limit của tài khoản đó, và không ai biết mình đã tiêu bao nhiêu.
Hub giải quyết bằng hai lớp credential và một quy tắc resolve.

Lớp	Ai sở hữu
Personal	Từng user upload .credentials.json của Claude CLI mình đang đăng nhập
Shared	Admin publish một credential ở scope workspace

Quy tắc resolve là **own → shared → none**: ưu tiên credential của chính user, fallback về shared, và trả về none một cách tường minh thay vì fail ngầm. PUT /credentials/claude/mode đổi mode; chip credential trên header phản ánh ngay.
Refresh token được ghi ngược về hub. Access token của Claude OAuth sống vài giờ, nên một .credentials.json thật gần như luôn quá expiresAt. Claude CLI tự renew từ refresh token trong lúc chạy, và agent PUT /credentials/claude/refreshed để hub giữ bản mới nhất. Hub track *sự tồn tại* của refresh token — cột boolean has_refresh_token, không phải chính token — nên một credential quá hạn nhưng renew được resolve thành status refreshable thay vì expired. Tín hiệu expired chỉ đến từ một nguồn: Claude CLI thật sự bị reject.
Usage tracking per-user. Mỗi call ghi token count và cost qua POST /credentials/claude/usage, roll up thành phần trăm của session limit và weekly limit, hiển thị trên chip credential.
Một hàm duy nhất trả ra credential material. resolve_material trong api/app/services/claude_credentials.py là hàm duy nhất trong hub trả credential material. Mọi endpoint khác khai báo response_model không có trường nào chứa được nó, nên một lỗi trong handler cũng không serialise nổi token ra response.

2. Spec sinh từ DOM thật

Cách thông thường để LLM sinh test automation: đọc source, suy ra selector, sinh file, chạy, fail, heal. Selector suy đoán là nguyên nhân chính của spec chết yếu — nó hợp lý trong code review và không tồn tại trên DOM thật.
Q-Agent có authoring mode **live-harness**: agent drive browser thật trước, emit spec sau. Qua CLI browser-harness gắn vào một Chrome đã authenticated:
1. **Thực thi từng step của test case trên app đang chạy.** Không mô phỏng.
2. **Resolve element qua accessibility tree**, ghi lại selector theo thứ tự ưu tiên cố định: data-testid → ARIA role + accessible name → label → CSS ổn định. Không :nth-child, không class trần, không structural combinator.

3. **Verify selector chứ không verify tọa độ.** `click_at_xy` trúng bất kỳ element nào nằm ở điểm đó — thường là một `<a>` bên trong — trong khi spec sẽ click **selector đã ghi**, mà tâm của nó có thể là container không handle click. Nên mỗi interaction được dispatch trực tiếp trên chính selector sắp emit, rồi verify hiệu ứng thật (URL có đổi không). Playwright `click()` pass bất cứ khi nào nó click trúng *một cái gì đó*, nên một navigation chưa được quan sát qua selector đó không tính là verified.
4. **Tạo test data nếu thiếu**, qua UI, và bake luôn setup đó vào spec để lần chạy sau tự đúng được.
5. **Emit spec** từ đúng những gì đã chạy.

Kết quả: một spec Playwright + TypeScript chạy green ngay, không cần heal pass.

Mode blind vẫn giữ lại. Sinh từ Knowledge Base + Project Config, rồi self-heal: feed failure kèm live DOM ngược lại, fix có mục tiêu, re-run — tối đa `heal_max_attempts = 3`, Playwright timeout rút ngắn, chạy trên model rẻ. Một heal pass có grounding DOM ghi entry **verified-at-runtime** ngược vào Knowledge Base.

Placeholder gate chặn trước execution. Spec sinh ra bị pre-flight check: selector bịa, TODO stub và URL placeholder đều bị chặn. Verdict passed / blocked / rejected. Spec blocked surface lên UI là *cần grounding* — thường là rebuild KB hoặc chạy một exploration pass — thay vì fail lặng lẽ lúc execution và bị đọc nhầm thành product defect.

Cả ba mode đều bounded bằng cost ceiling, turn cap và wall-clock timeout.

3. Knowledge Base build từ source

project-bootstrap traverse source thật của repository — clone nếu cần — để rút ra stack, kiến trúc, domain, route, selector, auth flow, environment, và các Page Object / fixture tái dùng được. Output là `knowledge.md` + `knowledge.json`, **per repository**.

Điều này đổi bản chất của prompt phía sau. requirement-analyst, test-case-generator và automation-generator không nhận mô tả chung chung; chúng nhận base URL, route, selector và test account thật của chính project đó.

KB **giàu lên theo thời gian**: mỗi selector verify được trên app đang chạy được stamp timestamp kèm strategy đã hoạt động, và entry verified-at-runtime thắng entry suy ra từ source, không bị merge sau ghi đè.

Một lượt build đầy đủ tốn khoảng 20 phút, nên hub hỗ trợ **clone**: admin build project mẫu trong shared namespace, member clone Project + ProjectConfig (kèm test account đã encrypt) + các ProjectKnowledge row và artefact trên đĩa (`knowledge/`, `repos/`, `auth/`) về scope của mình, re-stamp `owner_id`.

4. Ba approval gate do người giữ

- **Review Center.** Test case sinh ra ở trạng thái pending; chỉ case approved đi tiếp sang create-and-link và automation. Reviewer sửa được automation type của từng case (Playwright / Selenium / Cypress / Manual) — case Manual không bao giờ được đem sinh spec.
- **Create & Link có local mode.** Bước push test case lên provider chạy được ở chế độ dry-run: ghi local, không write lên provider.
- **Comment preview.** Kết quả trả về ticket là comment được chuẩn bị và preview trước, kèm status mapping cấu hình được, không phải một lần write thẳng.

Mỗi failure được execution-analyzer gán **failure class**: test_defect / product_defect / flaky / environment / timeout. Việc này giữ cho một spec sai không bị đọc thành product bug.

5. Kiến trúc bảo mật

Ba loại secret, ba boundary khác nhau, mỗi loại có một cơ chế cường chế riêng.

5.1. Provider PAT — không rời khỏi hub

Hub lưu PAT của Azure DevOps / Jira / GitHub encrypted at rest và **proxy mọi provider call**. Agent không bao giờ nhận PAT. Endpoint trả hasPat: true, không bao giờ trả PAT.

5.2. Claude credential — exception duy nhất, và nó được thu hẹp

Claude CLI cần credential trên disk mới chạy được, nên đây là secret duy nhất hub cố ý phát ra. Hai cơ chế giới hạn thiết hại:

Biến môi trường hẹp. Agent trả CLAUDE_SECURESTORAGE_CONFIG_DIR — biến chỉ relocate đúng file credential — chứ không phải CLAUDE_CONFIG_DIR, vốn kéo theo cả skills/, settings.json và projects/.

Run-scoped credential grant. Agent access token có TTL 15 phút, trong khi riêng Claude bootstrap của Q-Agent đã có timeout 1200 s và một run đầy đủ còn dài hơn — nên token không phải là cơ chế đúng cho background work. ADR 0009 đưa ra grant gắn với đúng một run, và grant đó **chỉ reach được ba route** trong toàn hub:

Route	Mục đích
GET /credentials/claude/resolve	Lấy credential để chạy
PUT /credentials/claude/refreshed	Ghi lại token CLI đã renew
POST /credentials/claude/usage	Ghi usage của một call đã xong

Giới hạn này **không** được cường chế bằng một if trong hàm xử lý grant, mà bằng chính wiring: không route nào khác trong hub depend vào require_credential_grant, và audience của grant không bao giờ registerable, nên require_principal và require_user đều reject nó. Thêm một route mới không vô tình mở rộng phạm vi của grant.

5.3. Browser session của tester — không rời khỏi máy tester

App sau SSO/MFA cần một người đăng nhập thật trong headed browser. **Local Agent** — app Node chạy trên máy tester — execute spec tại chỗ; cookie và storageState ở lại trên device, chỉ spec, kết quả và evidence đi ngược về server.

Device tự chứng minh identity qua **device pairing**: app mint một pairing code ngắn hạn, CLI đổi lấy token lâu dài per-device lưu tại ~/.qagent-agent/config.json, server chỉ giữ hash trên một row AgentDevice thuộc về user đó. Mọi agent job scope theo device owner; token revoke được từ app.

5.4. Hai key, không bao giờ dẫn xuất từ nhau

EMEHUD_JWT_SECRET ký JWT; EMEHUD_ENCRYPTION_KEY encrypt data at rest (ADR 0005). Thiếu một trong hai thì API **refuse to start** — không có fallback tự sinh, vì một encryption

key sinh lúc boot sẽ tạo ra các row không decrypt được sau restart kế tiếp. Q-Agent gộp hai giá trị này làm một; ADR 0005 tồn tại để không lặp lại.

5.5. Không fail-open

Không có configuration nào trong hub mà "authentication không khả dụng" dẫn tới "cho vào". Landing page theo cùng quy tắc: đọc lỗi product availability nghĩa là **đóng**. Riêng launch state degrade theo chiều ngược lại — một nút Launch chết tốt hơn một màn Overview trắng.
Mọi thao tác đáng ghi đều vào **audit log** append-only: category, actor + actor type, action, target, IP, status, run code.

6. Một session, một origin, ba ứng dụng

Hub mint access token audience-scoped cho đúng một agent, ký bằng key của hub; agent validate locally, **không call ngược về hub theo từng request** (ADR 0008).
Mỗi access token mang sid — session id — nên **revoke một session là log out device đó khỏi mọi agent**, không phải ba lần revoke ở ba nơi.
Cả suite nằm sau một origin, agent mount theo path (ADR 0010): hub.chuongnd.click và hub.chuongnd.click/qagent/. Agent thêm vào chiếm một path segment mới, không phải một origin mới.

7. Hub là source of truth, với một boundary được ghi rõ

Provider connection, project, repository, base URL, environment, test account, credential — khai báo một lần ở hub, agent đọc xuống (ADR 0001).
Boundary được giữ chặt: **hub chỉ build những artefact mà nó đã sở hữu toàn bộ input** — hôm nay là knowledge base, và không gì khác (ADR 0007). Hub không sinh test, không sinh code, không drive browser, không tạo PR. Một thay đổi thêm domain behaviour ngoài carve-out đó thuộc về agent.
Contract giữa hub và agent là một document thật — INTEGRATION.md — và đổi contract thì phải update nó trong cùng PR.

Con số

Đếm trực tiếp từ repo tại thời điểm viết:

Test backend EmeHub	788 hàm test
Test backend Q-Agent	1358 hàm test
API router của hub	14
Màn hình UI của hub chạy trên endpoint thật	11
Claude skill chuyên biệt trong Q-Agent	14
ADR đã accept trong hub	12

Mỗi skill là một `SKILL.md` — methodology, quality rule, output template — được backend inject làm **system prompt** cho đúng action đó, trong khi prompt của caller vẫn pin chính xác shape JSON mà backend parse.

Đọc tiếp

- `USER-GUIDE.md` — dùng sản phẩm, đầu tới cuối
- `INTEGRATION.md` — contract giữa hub và agent
- `CONTEXT.md` — vocabulary dùng chung